HostingLint Architecture

Version: 0.1.0

Date: February 2026

For: NLnet NGI Zero Commons Fund Application

System Overview

HostingLint is a static analysis engine that validates hosting control panel modules across three platforms using a unified TypeScript architecture.

```
┌──────────────────┐       ┌──────────────────────┐
│  CLI (hostinglint)    │       │  VS Code Extension      │
│  Commander.js + SARIF │       │  Diagnostics + Hover     │
└──────────────────┘       └──────────────────────┘
          │                           │
          └──────────────────────────┘
                     │
                     │
            ┌─────────▼─────────┐
            │  @hostinglint/core     │
            │  analyzeAuto() + API   │
            └─────────────────┘
                     │
                     │
     ┌───────────────┼──────────────────┐
     │               │                  │
┌────▼──────┐   ┌────▼──────┐   ┌──────────▼──────────┐
│ PHP        │   │ Perl        │   │ OpenPanel        │
│ WHMCS      │   │ cPanel      │   │ Docker extensions│
└──────────┘   └──────────┘   └──────────┬──────────┘
     │               │                  │
┌────▼──────┐   ┌────▼──────┐   ┌──────────▼──────────┐
│ PHP Rules  │   │ Perl Rules  │   │ OpenPanel Rules  │
│ + Common   │   │ + Common    │   │ + Common         │
└──────────┘   └──────────┘   └──────────────────┘
```

Package Structure

npm Workspace Monorepo

```
hostinglint/
├── packages/
│   ├── core/                @hostinglint/core (library)
│   │   ├── src/
```

```
│   │   │   ├── analyzers/    Platform-specific analysis engines
│   │   │   │   ├── base.ts   Base analyzer factory + version utils
│   │   │   │   ├── php/      PHP/WHMCS analyzer
│   │   │   │   ├── perl/     Perl/cPanel analyzer
│   │   │   │   └── openpanel/ OpenPanel/Docker analyzer
│   │   │   ├── rules/        Rule definitions
│   │   │   │   ├── php/      PHP compatibility, security, WHMCS
│   │   │   │   ├── perl/     Perl security, compatibility, best practice
│   │   │   │   ├── openpanel/ Docker security, compatibility
│   │   │   │   ├── common/   Cross-platform rules (credentials, eval, TODO)
│   │   │   │   ├── registry.ts  RuleRegistry class
│   │   │   │   └── index.ts  Rule exports and lookup functions
│   │   │   ├── analyze.ts     Unified analyzeAuto() entry point
│   │   │   ├── config.ts     Configuration loading/validation
│   │   │   ├── fixer.ts      Auto-fix engine
│   │   │   ├── inline-disable.ts  Inline comment directives
│   │   │   ├── plugins.ts    Plugin loading/validation
│   │   │   ├── presets.ts     Shared configurations
│   │   │   ├── types.ts      Core type definitions
│   │   │   └── index.ts      Public API
│   │   └── tests/            Vitest test suite (131 tests)
│   ├── cli/                  hostinglint CLI (Commander.js)
│   │   ├── src/cli.ts        Entry point
│   │   └── tests/            CLI integration tests
│   └── vscode/               VS Code extension (hostinglint-vscode)
│       ├── src/
│       │   ├── extension.ts  Activation + event listeners
│       │   ├── diagnostics.ts analyzeAuto() → Diagnostic conversion
│       │   ├── config.ts     Config cache + .hostinglintrc.json
│       │   ├── code-actions.ts Quick fixes + disable-line
│       │   ├── hover.ts      Rule info on hover
│       │   ├── logger.ts     OutputChannel logging
│       │   └── constants.ts  Shared constants
│       ├── tests/            Vitest tests with vscode mock
│       └── esbuild.mjs       CJS bundle for extension host
├── examples/                 Sample modules for testing
│   ├── whmcs-sample/         Clean WHMCS modules
│   ├── cpanel-sample/        Clean cPanel plugins
│   └── vulnerable/           Intentionally vulnerable examples
│       ├── whmcs-vulnerable/
│       ├── cpanel-vulnerable/
│       └── openpanel-vulnerable/
└── docs/                     Documentation
```

Analysis Pipeline

Data Flow

```
Input File  →  analyzeAuto()  →  Select Rules  →  Run Checks  →  Apply Overrides  →
Output
```

```
   .php      detect: WHMCS      PHP rules         regex match     rule overrides
text/json/sarif
   .pl       detect: cPanel     Perl rules        per-line scan   severity changes
vscode diagnostics
   Dockerfile detect: OpenPanel  OpenPanel rules   + context       'off' filtering
```

The unified `analyzeAuto()` entry point (in `packages/core/src/analyze.ts`) handles platform detection, analyzer dispatch, and rule overrides. Both the CLI and VS Code extension use this single function.

Analyzer Factory Pattern

All three analyzers share a common factory (`createAnalyzer`) that:

1. Merges user options with defaults
2. Filters rules based on platform, severity, and version constraints
3. Constructs a `RuleContext` with pre-split lines and config
4. Runs each applicable rule against the code
5. Sorts results by line number and column
6. Returns `LintResult[]`

Version filtering is platform-specific:

- PHP rules filter by `minPhpVersion` (e.g., `php-compat-each` only fires for PHP >= 8.0)
- Perl rules filter by `minCpanelVersion`
- OpenPanel rules have no version constraints (always active)

Rule Architecture

Rule Interface

```
interface Rule {
  id: string;                           // e.g., 'security-sql-injection'
  description: string;                  // Human-readable
  severity: 'error' | 'warning' | 'info';
  category: string;                     // 'security', 'compatibility', 'best-practice'
  platform: 'whmcs' | 'cpanel' | 'openpanel' | 'all';
  check: (context: RuleContext) => LintResult[];
  minPhpVersion?: PhpVersion;           // Version constraint
  minWhmcsVersion?: WhmcsVersion;
  minCpanelVersion?: CpanelVersion;
}
```

Detection Approach

All rules use regex-based pattern matching on code strings:

- **Simple patterns** (90%): Single-line regex matching (e.g., `\beach\s*\(`)
- **Context-aware** (8%): Multi-line window analysis (check surrounding lines)
- **Structural** (2%): File-level analysis (check for presence/absence of constructs)

No PHP or Perl runtime is required. No code is ever executed.

False Positive Mitigation

1. Comment detection: skip lines starting with `//`, `#`, `/*`, `*`
2. String exclusion: patterns in quoted strings are filtered
3. File-type validation: rules only run on relevant file types
4. Context validation: additional evidence required (e.g., WHMCS module structure)

Output Formats

Format	Use Case	Standard
text	Human-readable CLI output	ESLint-style
json	Machine processing, piping	JSON array
sarif	GitHub Code Scanning, CI/CD	SARIF 2.1.0

Configuration System

.hostinglintrc.json

```
{
  "rules": { "whmcs-license-check": "off" },
  "phpVersion": "8.1",
  "ignore": ["vendor/**"],
  "preset": "recommended"
}
```

Inline Disabling

```
// hostinglint-disable-next-line security-sql-injection
// hostinglint-disable rule-id
// hostinglint-enable rule-id
```

Presets

- **recommended**: balanced error/warning configuration
- **strict**: all rules at error level
- **security-only**: only security rules enabled

VS Code Extension

Architecture: Direct API (no LSP)

The extension calls `analyzeAuto()` from `@hostinglint/core` directly in the VS Code extension host process. Since core analysis is synchronous and sub-millisecond per file, a Language Server Protocol layer is unnecessary overhead.

Module Overview

Module	Responsibility
`extension.ts`	Lifecycle (activate/deactivate), event listeners, provider registration
`diagnostics.ts`	Calls `analyzeAuto()`, converts `LintResult[]` → `vscode.Diagnostic[]`, maintains result store
`config.ts`	Config cache per workspace folder, reads `.hostinglintrc.json` via core's `findConfig()`, falls back to VS Code settings
`code-actions.ts`	Quick fixes from `fixAction`, "disable rule for this line" for PHP/Perl
`hover.ts`	Rule details on hover (ID, category, platform, severity, fix suggestion)
`logger.ts`	`OutputChannel('HostingLint')` for error logging

Key Design Decisions

- **Config cache per workspace folder** — `findConfig()` result is cached by workspace folder URI, invalidated on `.hostinglintrc.json` change via `FileSystemWatcher`
- **Result store (`Map<string, LintResult[]>`)** — Keyed by document URI, used by hover and code actions to look up full rule info without re-analyzing
- **Debounced onType analysis** — 300ms debounce prevents excessive analysis during rapid typing
- **End-of-line diagnostic range** — Underlines extend from the issue column to end of line, avoiding word-boundary ambiguity
- **Disable-line for PHP/Perl only** — Dockerfiles and YAML have no inline comment standard for disable directives
- **`setTimeout(fn, 0)` batching** — Config change re-analysis of all open documents is batched to avoid blocking

Build

The extension is bundled with esbuild into a single CJS file (`dist/extension.js`), with `vscode` as an external dependency. The bundle is ~72KB.

Performance

- Analysis is sub-millisecond per file (0.13 ms average, benchmarked)
- Target: < 100 ms per 1000 lines with 50+ rules
- Regex patterns are compiled once, reused across lines
- Early returns skip irrelevant files (e.g., `.php` rules skip `.pl` files)

Security Model

- **Read-only**: HostingLint never modifies analyzed files
- **No execution**: Code is analyzed as strings, never evaluated
- **No runtime**: No PHP or Perl interpreter required
- **Pure functions**: Rules have no side effects, no network calls
- **Sandboxed plugins**: Plugin rules validated against strict interface

---

Prepared for NLnet NGI Zero Commons Fund Application, February 2026